

CS714: Secure Computation

Adithya Vadapalli

IIT Kanpur

March 13, 2026

- The protocol described in Chapter 3 is:
 - perfectly correct
 - perfectly private
- However, it assumes that *all parties follow the protocol exactly*.
- In practice, some parties may deviate from the protocol.
- Later we will study protocols that remain:
 - correct
 - privateeven when some parties deviate.
- Such protocols are called *robust protocols*.

Defining Privacy

Recall the informal definition from Chapter 3.

- Consider inputs:

$$(x_1, x_2, \dots, x_n)$$

- Let

$$C \subseteq \{P_1, \dots, P_n\}, \quad |C| \leq t$$

be the set of corrupted parties.

- For each corrupted party P_j , define view_j the *entire view of party P_j* during the execution.
- The collection $\{\text{view}_j\}_{P_j \in C}$ represents the *information leaked to corrupted parties*.

Example 4.1: Influence in Protocol CEPS

Setting

- Protocol: CEPS (Circuit Evaluation with Passive Security)
- Parties: P_1, P_2, P_3
- Corruption threshold: $t = 1$
- Function:

$$f(x_1, x_2, x_3) = (x_1 x_2 x_3, x_2, x_3)$$

Example 4.1: Influence in Protocol CEPS

Setting

- Protocol: CEPS (Circuit Evaluation with Passive Security)
- Parties: P_1, P_2, P_3
- Corruption threshold: $t = 1$
- Function:

$$f(x_1, x_2, x_3) = (x_1 x_2 x_3, x_2, x_3)$$

Goal

- P_1 should learn $x_1 x_2 x_3$
- P_2 learns x_2
- P_3 learns x_3

Example 4.1: Corrupted P_1 Influencing the Protocol

Attack

- Assume P_1 is corrupted.
- Instead of its real input x_1 , it uses

$$x'_1 = 1$$

- P_1 secret shares 1 during the input sharing phase.
- Then it follows the rest of the protocol honestly.

Example 4.1: Corrupted P_1 Influencing the Protocol

Attack

- Assume P_1 is corrupted.
- Instead of its real input x_1 , it uses

$$x'_1 = 1$$

- P_1 secret shares 1 during the input sharing phase.
- Then it follows the rest of the protocol honestly.

Result

$$y_1 = x'_1 x_2 x_3$$

$$y_1 = 1 \cdot x_2 x_3 = x_2 x_3$$

Example 4.1: Corrupted P_1 Influencing the Protocol

Attack

- Assume P_1 is corrupted.
- Instead of its real input x_1 , it uses

$$x'_1 = 1$$

- P_1 secret shares 1 during the input sharing phase.
- Then it follows the rest of the protocol honestly.

Result

$$y_1 = x'_1 x_2 x_3$$

$$y_1 = 1 \cdot x_2 x_3 = x_2 x_3$$

Observation

- P_1 learns $x_2 x_3$
- This value is independent of its real input x_1

Example 4.1: Key Question

What happened?

- P_1 deviates by choosing a different input.
- Protocol execution remains otherwise honest.

Example 4.1: Key Question

What happened?

- P_1 deviates by choosing a different input.
- Protocol execution remains otherwise honest.

Key Insight

- The adversary can **influence the output**.
- However, this does not necessarily violate security:
 - Parties are allowed to choose arbitrary inputs.

Example 4.1: Key Question

What happened?

- P_1 deviates by choosing a different input.
- Protocol execution remains otherwise honest.

Key Insight

- The adversary can **influence the output**.
- However, this does not necessarily violate security:
 - Parties are allowed to choose arbitrary inputs.

Takeaway

Passive security allows input manipulation but prevents learning extra information.

Example 4.2: Modified Circuit Evaluation

Protocol Setup

- Same protocol: CEPS
- Parties: P_1, P_2, P_3
- Corruption threshold: $t = 1$

Example 4.2: Modified Circuit Evaluation

Protocol Setup

- Same protocol: CEPS
- Parties: P_1, P_2, P_3
- Corruption threshold: $t = 1$

Circuit Evaluation Order

- 1 Compute multiplication gate

$$[u]_t = [x_1 x_2]_t$$

- 2 Compute second multiplication

$$[y_1]_t = [u x_3]_t$$

- 3 Reconstruct $[y_1]_t$ to P_1

Example 4.2: Corrupted Party Behavior

Adversarial Setup

- Assume P_1 is corrupted
- Uses modified input

$$x'_1 = 0$$

Example 4.2: Corrupted Party Behavior

Adversarial Setup

- Assume P_1 is corrupted
- Uses modified input

$$x'_1 = 0$$

Multiplication Step

Players compute polynomial

$$h(0) = x'_1 x_2$$

Example 4.2: Corrupted Party Behavior

Adversarial Setup

- Assume P_1 is corrupted
- Uses modified input

$$x'_1 = 0$$

Multiplication Step

Players compute polynomial

$$h(0) = x'_1 x_2$$

Instead of sending the correct share:

$$[h(1)]_t$$

P_1 sends

$$[r_1^{-1} + h(1)]_t$$

Example 4.2: Effect of the Deviation

Reconstruction becomes

$$[u]_t = r_1[r_1^{-1} + h(1)]_t + \sum_{i=2}^n r_i[h(i)]_t$$

Example 4.2: Effect of the Deviation

Reconstruction becomes

$$[u]_t = r_1[r_1^{-1} + h(1)]_t + \sum_{i=2}^n r_i[h(i)]_t$$

Simplifying:

$$= [1 + \sum_{i=1}^n r_i h(i)]_t$$

Example 4.2: Effect of the Deviation

Reconstruction becomes

$$[u]_t = r_1[r_1^{-1} + h(1)]_t + \sum_{i=2}^n r_i[h(i)]_t$$

Simplifying:

$$\begin{aligned} &= [1 + \sum_{i=1}^n r_i h(i)]_t \\ &= [1 + x'_1 x_2]_t \end{aligned}$$

Example 4.2: Effect of the Deviation

Reconstruction becomes

$$[u]_t = r_1[r_1^{-1} + h(1)]_t + \sum_{i=2}^n r_i[h(i)]_t$$

Simplifying:

$$\begin{aligned} &= [1 + \sum_{i=1}^n r_i h(i)]_t \\ &= [1 + x'_1 x_2]_t \end{aligned}$$

Since $x'_1 = 0$:

$$[u]_t = [1]_t$$

Example 4.2: Final Outcome

Next multiplication:

$$[y_1]_t = [ux_3]_t$$

Since $u = 1$:

$$y_1 = 1 \cdot x_3$$

Example 4.2: Final Outcome

Next multiplication:

$$[y_1]_t = [ux_3]_t$$

Since $u = 1$:

$$y_1 = 1 \cdot x_3$$

$$y_1 = x_3$$

Example 4.2: Final Outcome

Next multiplication:

$$[y_1]_t = [ux_3]_t$$

Since $u = 1$:

$$y_1 = 1 \cdot x_3$$

$$y_1 = x_3$$

Observation

- P_1 learns x_3
- This is **more information than allowed**

Example 4.2: Security Insight

What went wrong?

- P_1 manipulated intermediate shares.
- This changed the value of u .

Example 4.2: Security Insight

What went wrong?

- P_1 manipulated intermediate shares.
- This changed the value of u .

Result

- The circuit evaluation becomes incorrect.
- The adversary learns extra information.

Example 4.2: Security Insight

What went wrong?

- P_1 manipulated intermediate shares.
- This changed the value of u .

Result

- The circuit evaluation becomes incorrect.
- The adversary learns extra information.

Lesson

Passive security does not protect against malicious deviations.

Universally Composable (UC) Model

- To reason about robustness formally, we need a precise *security model*.
- We will use the *UC (Universally Composable) security model*.
- The key idea:

Universal Composability

A protocol proven secure in the UC model remains secure regardless of the context in which it is used.

- In other words, a UC-secure protocol can be composed with
 - other protocols
 - arbitrary environmentswhile preserving security.
- Our formulation will differ slightly from the original UC formalization.

Allowed vs Leaked Information

- Corrupted parties must necessarily learn:

$$\{x_j, y_j\}_{P_j \in C}$$

where

- x_j = their inputs
- y_j = their outputs
- These are called the *allowed values*.
- The protocol is secure if the *leaked values* reveal no additional information.

Privacy Definition

A protocol is private if the leaked values contain no more information than the allowed values.

Computational View of Privacy

We want to formalize:

Does the leaked information reveal more than the allowed information?

A natural approach:

Idea

If the leaked values can be computed from the allowed values, then they reveal no additional information.

However, this must consider *computational efficiency*.

- Example: factoring large integers
- Knowing $n = pq$ does not efficiently reveal (p, q)

Thus we require *efficient computability*.

Refined Definition of Privacy

Following this idea, we obtain a stronger definition.

Definition

A protocol is private if the leaked values can be computed efficiently from the allowed values.

Intuition:

- The adversary's view should reveal nothing beyond:
 - its inputs
 - its outputs
- If the leaked values can be generated from allowed values, then the execution reveals no additional information.

The Simulator Idea

To prove privacy, we show that an algorithm exists that generates the adversary's view.

Simulator

An efficient algorithm that generates the leaked values using only the allowed values.

Important observation:

- Protocol executions involve randomness.
- Therefore the adversary's view is a *random variable*.

The simulator must therefore generate a distribution of views.

Definition

There exists an efficient simulator S such that

$$S(\{x_j, y_j\}_{P_j \in C})$$

and

$$\{\text{view}_j\}_{P_j \in C}$$

have the same distribution.

Interpretation:

- The adversary's view can be simulated using only
 - corrupted parties' inputs
 - corrupted parties' outputs
- Therefore the execution leaks no additional information.

Defining Robustness

Privacy concerns *what the adversary learns*.

Robustness concerns *how the adversary can influence the protocol*.

Goal

Ensure the protocol behaves correctly even if corrupted parties deviate.

Key assumption:

- Corrupted parties are *Byzantine*.
- The adversary may instruct them to send arbitrary messages.

Thus the protocol must remain correct regardless of adversarial behavior.

Example: Influence in MPC

Consider circuit evaluation with passive security.

Parties compute:

$$f(x_1, x_2, x_3)$$

Suppose P_1 is corrupted.

Attack:

- During input sharing, P_1 replaces input x_1 with 1.
- Then the protocol proceeds honestly.

Result:

- P_1 learns

$$y_1 = x_2 x_3$$

This shows a party may influence what it learns.

Robustness asks:

How much can an adversary influence the outcome?

In the previous example:

- The corrupted party manipulated the computation
- The final output changed

Robust MPC protocols must:

- restrict adversarial influence
- ensure correct outputs for honest parties

Influence in the Previous Example

The previous example showed that party P_1 could manipulate the protocol so that it always learns x_3 .

Question:

Is this a security problem?

Observation:

- If an adversary controls a party, it can change that party's input.
- This is called **input substitution**.

Example:

$$f(x_1, x_2, x_3)$$

If P_1 replaces x_1 with x'_1 , the protocol computes

$$f(x'_1, x_2, x_3)$$

This influence is unavoidable.

Allowed Influence

To reason about robustness, we define an **ideal secure evaluation**.

Imagine an ideal functionality:

- Each party submits its input to a trusted box.
- The box computes

$$(y_1, \dots, y_n) = f(x_1, \dots, x_n)$$

- Each party receives its output.

If a party is corrupted:

- it may submit a different input x'_i

This is the only influence permitted in the ideal world.

Allowed Influence

Changing one's own input.

Actual Influence

In the real protocol execution:

- Corrupted parties may send arbitrary messages
- They may deviate from the protocol

This is called the adversary's

actual influence

Goal of robustness:

Robustness Goal

Any effect caused by the adversary in the real protocol should correspond to some allowed influence.

Definition

A protocol is robust if the effect of any actual influence can also be obtained using an allowed influence.

Interpretation:

- Even if the adversary deviates arbitrarily,
- the result should be equivalent to
- simply changing its input.

Thus the adversary cannot create effects beyond input substitution.

Simulation-Based Robustness

We formalize robustness using simulation.

Definition

A protocol is robust if there exists an efficient simulator S such that for every adversary attacking the protocol, S can efficiently compute an allowed influence with the same effect.

Interpretation:

- Whatever the adversary achieves in the real protocol
- can be reproduced in the ideal model

Thus the adversary gains no additional power.

Privacy and Robustness

The previous attack appears to affect only correctness.

However:

- Incorrect outputs may leak information.
- Thus correctness violations may also break privacy.

Example:

- Manipulating intermediate values
- May reveal private inputs.

Key Insight

Privacy and correctness cannot be treated independently.

They must be defined together.

Combining Privacy and Robustness

The final definition requires: **one simulator handling both properties**

The simulator must:

- translate adversarial behavior into allowed influence
- simulate the adversary's view

If this is possible:

- anything the adversary can achieve in the real protocol
- can also occur in the ideal execution.

Sketch of UC Security

To formalize these ideas we use the **Universally Composable (UC) model**.

The UC framework models protocols as:

- interactive computational entities
- that communicate by sending messages

These entities are called

interactive machines

Security is defined by comparing:

- a real protocol execution
- an ideal functionality

In the UC model:

- Each entity has **ports**
- Ports represent communication channels.

If machine A_1 outputs message m on output port P , and machine A_2 has input port P , then m becomes the input to A_2 .

This forms a network of interacting machines.

In the UC framework, each participant is modeled as an **interactive agent**.

- An interactive agent is like a computer program.
- It has several communication **ports**.
- It can:
 - receive messages on input ports
 - send messages on output ports

Key property:

- The agent only changes its internal state when activated.

Each activation corresponds to processing a message.

Activation of Agents

Execution proceeds through **activations**.

- An activation specifies a port where a message arrives.
- The agent:
 - reads messages from its input ports
 - updates its internal state
 - sends messages on output ports

The agent also specifies a **return port**.

- This determines which agent is activated next.

Thus execution moves from one agent to another.

An **interactive system** consists of multiple agents.

- Agents communicate via ports.
- Each port connects exactly one output to one input.

When agent A sends message m on port P :

- m is placed in the message queue of port P
- the connected agent receives the message.

This models a distributed protocol execution.

Open Ports and the Environment

Some ports may not be connected to another agent.

These are called **open ports**.

- Messages arriving at open input ports come from the environment.
- Messages sent to open output ports are visible externally.

Thus the environment interacts with the protocol through these open ports.

Behavioral Equivalence

When analyzing protocols, we do not care about internal structure.

We only observe:

- inputs on open ports
- outputs on open ports

Behavioral Equivalence

Two systems are equivalent if they produce the same outputs for the same inputs.

Such systems are called

behaviorally equivalent.

Security by Behavioral Equivalence

UC security compares two systems:

- the **real protocol**
- an **ideal functionality**

The ideal functionality describes the desired behavior.

Security requirement:

UC Security Idea

The real protocol must be behaviorally equivalent to the ideal functionality.

Thus an external observer cannot distinguish between them.

Ideal Functionality

The ideal functionality F specifies how the protocol should behave.

Example: secure function evaluation

- 1 Each party submits input x_i
- 2 The functionality computes

$$(y_1, \dots, y_n) = f(x_1, \dots, x_n)$$

- 3 Each party receives output y_i

The functionality may leak only **allowed information**.

Example: Ideal Secure Computation

An ideal functionality for secure computation:

- 1 Receive input x_i from each party P_i
- 2 Compute

$$(y_1, \dots, y_n) = f(x_1, \dots, x_n)$$

- 3 Output y_i to party P_i

Possible leakage:

- when parties submit inputs
- when outputs are delivered

These correspond to allowed leakage in the UC model.

Ideal Functionality vs Real Protocol

Suppose an ideal functionality F existed in the real world.

- Each party sends input x_i to F .
- F computes

$$(y_1, \dots, y_n) = f(x_1, \dots, x_n)$$

- Each party receives output y_i .

Properties:

- Correct outputs
- Only allowed leakage
- Limited adversarial influence

This represents the **ideal execution**.

Secure Implementation

A real protocol π should behave like the ideal functionality F .

Definition

Protocol π securely implements functionality F if its behavior is indistinguishable from F .

Interpretation:

- Running π should look the same as interacting with F .

Thus:

$$\pi \approx F$$

from the viewpoint of any external observer.

The Simulator

Problem:

- The real protocol exposes many ports.
- The ideal functionality has very limited interaction.

Solution: introduce a **simulator** S .

Role of S :

- interact with the ideal functionality
- simulate the protocol execution
- reproduce the adversary's view

Goal

Make the real and ideal executions indistinguishable.

Real vs Ideal Worlds

Real execution

- Parties run protocol π
- Adversary interacts directly with them
- Communication network is explicit

Ideal execution

- Parties interact with functionality F
- Simulator S interacts with the adversary
- S translates adversarial behavior

Security requires these two worlds to be **behaviorally equivalent**.

A key property of the UC framework:

UC Theorem

If protocol π securely realizes functionality F , then π can replace F in any larger protocol.

Implication:

- Complex protocols can be built modularly.
- Subprotocols can be analyzed independently.

Thus secure components remain secure when composed with other protocols.

Suppose a protocol π_{complex} uses functionality F .

The power of **UC composition**

Steps to prove security:

- 1 Show π securely implements F .
- 2 Show π_{complex} is secure using F .
- 3 Then: π_{complex} remains secure when F is replaced with π .

A **clocked entity** is an interactive agent with additional scheduling rules.

Properties:

- receives activation tokens
- processes incoming messages
- outputs messages on ports

This mechanism controls the order in which agents execute.

Leakage and Influence Ports

Clocked entities in the UC model have special ports:

- **Influence ports** (e.g., $N.infl$)
- **Leakage ports** (e.g., $N.leak$)

Purpose:

- Influence ports allow adversarial control.
- Leakage ports expose allowed information.

These ports are used to control

- adversarial influence
- allowed information leakage.

Clocked Entities

Each entity maintains two state variables:

- **active** $\in \{0, 1\}$
- **calling** $\in \{0, 1\}$

Initially: active = 0, calling = 0

Interpretation:

- active = 1 $\rightarrow$ entity currently executing
- calling = 1 $\rightarrow$ entity made a recursive call

These variables regulate activation flow.

When an inactive entity receives an activation token:

- If the port is an influence port:
 - the entity becomes active
 - it begins executing
- Otherwise:
 - the token is returned immediately

This prevents unintended execution.

Recursive Calls

A clocked entity may call another entity.

Steps:

- ① Send activation token on an influence port.
- ② Record the return port.
- ③ Wait for the called entity to return control.

When control returns:

- the calling entity resumes execution.

This mechanism models nested protocol calls.

Returning Control

Two types of returns exist:

Return on clock

- Entity finishes execution.
- Activation token is returned to caller.

Return from recursive call

- Called entity returns activation token.
- Caller resumes execution.

This ensures a well-defined order of execution.

An interactive system may consist of several clocked entities.

Lemma

A composition of clocked entities is itself a clocked entity.

Implication:

- complex systems can be built from smaller components
- execution rules remain consistent

This supports modular protocol design in the UC framework.

Activation Flow Example

Example execution flow:

$$S \rightarrow T \rightarrow N$$

- S calls T
- T calls N
- N performs its computation
- control returns to T
- control returns to S

This recursive structure models nested protocol execution.

Security by Behavioral Equivalence

A protocol is considered secure if its execution in the real world behaves indistinguishably from an ideal system that performs the intended task.

Real System

- Parties execute a protocol π .
- Communication takes place over a network.
- An adversary may interfere with communication.

Ideal System

- Parties interact with an ideal functionality F .
- F computes the intended function.
- Only explicitly allowed leakage and adversarial influence occur.

Security means that no external observer (called the environment) can distinguish whether it is interacting with the real system or the ideal one.

Modeling the Real System

In the UC framework, protocol execution is modeled as an **interactive system** composed of multiple agents that communicate through ports.

The system contains the following components.

Parties

Each party P_i runs the protocol.

Each party has two main ports:

- $P_i.in$: an input port through which the party receives input.
- $P_i.out$: an output port through which the party delivers results.

Communication Resource

Instead of sending messages directly to each other, parties send messages through a resource R .

The resource R models the communication network and captures how messages move between parties.

This allows the UC model to describe network behavior and adversarial interference in a precise and modular way.

Role of the Communication Resource

The communication resource R represents the network connecting the parties. Its responsibilities include:

Message forwarding

Messages sent by one party are delivered to the intended recipient.

Leakage

Certain information about communication may be visible to the adversary. Examples include:

- message contents,
- message length,
- timing of communication,
- sender identity.

Role of the Communication Resource

The communication resource R represents the network connecting the parties. Its responsibilities include:

Adversarial influence

The adversary may influence network behavior, for example by:

- delaying messages,
- changing delivery order,
- scheduling when messages arrive.

By modeling the network as a separate resource, UC can precisely specify what information leaks and what control the adversary has.

Example: Authenticated Channel Without Order Preservation

Example 4.3 describes a communication resource modeling an authenticated channel. Such a channel provides authentication but does not guarantee that messages arrive in the order they were sent.

The properties of this channel are:

- Messages cannot be modified by the adversary.
- The identity of the sender is preserved.
- Messages may be delivered in any order.

To describe these properties, the resource R includes special ports that model leakage and adversarial influence.

These ports allow the UC framework to describe exactly what the adversary can observe and what it can control.

Leakage Port

The communication resource includes an output port $R.\textit{leak}$

- 1 This port outputs all messages that are sent through the resource.
- 2 This models the fact that an authenticated channel does not necessarily hide the contents of messages.
- 3 In many real-world settings, the adversary can observe network traffic even if it cannot modify messages.
- 4 For example, an adversary monitoring the network might see all transmitted messages while still being unable to alter them.
- 5 The information revealed through this port is called **allowed leakage**.

The communication resource also includes an input port $R.infl$. Through this port, the adversary can influence how messages are delivered.

For example, the adversary may specify:

- which message should be delivered next,
- when a message should be delivered,
- the order in which multiple messages are delivered.

This models the adversary's control over the communication network.

Even though the adversary cannot modify or forge messages, it can still affect the timing and ordering of communication.

Why Model Leakage and Influence Explicitly?

In the UC framework, the real and ideal systems must expose the same observable behavior to the environment.

To ensure this, we explicitly specify two aspects of the system:

- **Leakage**

Information that the adversary is allowed to learn from the system.

- **Influence**

Actions that the adversary is allowed to perform on the system.

If the ideal system revealed less information or allowed less control than the real protocol, the environment could distinguish the two executions.

Therefore, the ideal functionality must explicitly include the same leakage and influence that occur in the real system.

Ideal Functionality

To describe how a protocol is intended to behave, the UC framework introduces an **ideal functionality**.

The ideal functionality is denoted by F and represents a trusted party that performs the desired computation.

Its purpose is to provide a clear specification of the correct behavior.

The ideal functionality:

- receives inputs from parties,
- computes the intended function,
- sends outputs to the parties.

The internal protocol details are ignored; only the input-output behavior is important.

Example: Secure Function Evaluation

Consider a secure function evaluation problem.
Suppose n parties want to compute a function

$$f(x_1, x_2, \dots, x_n)$$

Each party P_i provides an input value

$$x_i$$

The functionality computes

$$(y_1, \dots, y_n) = f(x_1, \dots, x_n)$$

Each party P_i learns its corresponding output y_i .

The ideal functionality models this entire computation as if it were performed by a trusted third party.

Operation of the Ideal Functionality

The ideal functionality F works as follows.

Receiving Inputs

Each party P_i sends its input x_i to the functionality through the port

in_i

Computation

After receiving the required inputs, the functionality computes

$$(y_1, \dots, y_n) = f(x_1, \dots, x_n)$$

Output Delivery

The result y_i is sent to party P_i through the port

out_i

This models the ideal situation in which a trusted entity performs the computation correctly.

Leakage in the Ideal Functionality

The ideal functionality may still reveal limited information.
In this example, it reveals when a party provides its input.
When party P_i submits input x_i , the functionality outputs the message

“party i gave input”

on the port

$F.leak$

This models the fact that in real protocols the adversary often observes when parties send messages or participate in the protocol.
Allowing this leakage ensures that the ideal execution reflects realistic network observations.

Adversarial Influence on Output Delivery

The adversary may also control when outputs are delivered to the parties.
To model this, the ideal functionality includes an input port

$F.infl$

The adversary may send an instruction such as

deliver to party i

When the functionality receives this instruction, it outputs the value y_i to party P_i on the port out_i , provided that the output has not already been delivered.

This models the adversary's ability to delay or schedule message delivery in an asynchronous network.

Security Goal in the UC Framework

A protocol π is said to securely realize an ideal functionality F if the real execution and the ideal execution are indistinguishable.

Formally, the requirement is

$$Real(\pi, A, Z) \approx Ideal(F, S, Z)$$

where

- A is the real-world adversary,
- S is a simulator in the ideal world,
- Z is the environment attempting to distinguish the executions.

The simulator reproduces the adversary's view using only the information available in the ideal functionality.

If the environment cannot distinguish the two worlds, the protocol is considered UC-secure.

Recursive Polytime Clocked Entities

In the UC framework, every interactive agent is modeled as a **clocked entity**. To ensure that protocols remain efficient, the framework restricts the computational complexity of these entities.

A clocked entity is said to be **recursive polytime** if the following conditions hold:

- Each activation of the entity performs only expected polynomial-time computation in the security parameter k .
- The entity performs only an expected polynomial number of recursive calls before returning control.

This restriction guarantees that the entire system remains computationally efficient and prevents infinite recursive activations during protocol execution.

Composition of Recursive Polytime Entities

In a UC execution, multiple interactive agents are composed to form a larger interactive system.

Lemma

A composition of recursive polytime clocked entities is itself a recursive polytime clocked entity.

- 1 This result ensures that when we combine multiple protocol components, the overall system still runs within polynomial time.
- 2 Therefore, efficiency is preserved even when complex protocols are constructed by combining many smaller components.

Intuition Behind the Lemma

The proof idea relies on the following observations.

- Each individual agent performs only polynomial-time computation during each activation.
- The number of recursive activations within each agent is also bounded by a polynomial.
- When multiple agents are composed, activations pass between them, but the total number of activations remains polynomial.

Thus, the total expected running time of the composed system is bounded by a polynomial in the security parameter.

This guarantees that the UC framework models only efficient protocols.

Clocked Agents in the UC Framework

Several types of entities appear in the UC model, including:

- parties executing the protocol,
- ideal functionalities,
- simulators,
- adversaries.

All of these entities are modeled as **clocked agents**.

Whenever a clocked agent becomes active, it performs the following steps:

- reads messages from its incoming message queues,
- processes these messages according to its specification,
- possibly sends new messages,
- returns control once its computation is complete.

This activation model provides a precise description of protocol execution.

Ideal Functionalities

An **ideal functionality** is modeled as an interactive agent.

Each functionality has a unique name F and communicates through a well-defined set of ports.

If the functionality serves n parties, it has:

- n input ports

$$F.in_1, \dots, F.in_n$$

- n output ports

$$F.out_1, \dots, F.out_n$$

These ports are called the **protocol ports** because they represent communication with the protocol participants.

Special Ports of an Ideal Functionality

In addition to the protocol ports, an ideal functionality includes two special ports.

Influence Port

$F.infl$

This port allows the adversary to influence the behavior of the functionality, for example by controlling message delivery.

Leakage Port

$F.leak$

This port outputs information that the functionality is allowed to reveal to the adversary. These ports explicitly model the adversary's capabilities within the ideal system.

Example: Ideal Functionality for Secure Transfer

As an example, consider an ideal functionality F_{ST} for secure message transfer.

The functionality receives messages from a sender and eventually delivers them to a receiver.

The influence port determines the order in which messages are delivered.

The leakage port reveals limited information about the message, such as:

- the identities of the sender and receiver,
- the identifier of the message,
- the length of the message.

Purpose of the Leakage Specification

The leakage port reveals information of the form

$$(mid, i, j, |m|)$$

where

- mid is a message identifier,
- i is the sender,
- j is the receiver,
- $|m|$ is the message length.

This means that even in the ideal world the adversary learns certain metadata about communication.

Such leakage models realistic network observations, where an adversary may see message sizes or communication patterns even if the message contents remain hidden.

Ideal Functionality for Secure Transfer

We consider an ideal functionality F_{ST} that models secure message transfer between n parties. The functionality provides a trusted service that stores messages sent by parties and later delivers them.

The functionality communicates with parties through protocol ports:

- $ST.in_i$: input port for messages sent by party i
- $ST.out_i$: output port delivering messages to party i

In addition, it contains special ports:

- $ST.leak$: leakage port revealing allowed information
- $ST.infl$: influence port allowing adversarial control

These ports explicitly model what the adversary can observe and what it can control.

Behavior of the Secure Transfer Functionality

When the functionality receives a message from a sender, it performs two actions:

- It records the message internally.
- It leaks limited information about the message to the adversary.

More precisely:

- On input (mid, i, j, m) on $ST.in_i$ (where j is the receiver),
- The functionality outputs $(mid, i, j, |m|)$ on $ST.leak$.
- It then stores the tuple (mid, i, j, m) internally.

The identifier mid distinguishes multiple messages sent between the same pair of parties.

Delivery of Messages

The delivery of messages is controlled through the influence port.

- On input $(deliver, mid, i, j)$ on $ST.infl$,
- If a stored tuple (mid, i, j, m) exists, the functionality deletes the stored message.
- It then outputs (mid, i, m) on $ST.out_j$.

This models a network in which the adversary can delay or reorder message delivery but cannot modify messages.

Why Leak the Message Length?

The leakage port outputs information of the form

$$(mid, i, j, |m|)$$

where $|m|$ denotes the length of the message.

This models realistic network behavior.

In practice, many communication channels reveal metadata such as:

- who communicates with whom,
- when communication occurs,
- how large the message is.

Even if the contents of a message are encrypted, its size may still be visible.

Ideal functionalities also model how an adversary interacts with corrupted parties.

The functionality specifies what information is revealed and how control changes when a party becomes corrupted.

Two types of corruption are commonly considered:

- Passive corruption
- Active corruption

These models capture different levels of adversarial power.

Passive Corruption

Passive corruption models an adversary that observes the internal state of a party but does not alter its behavior.

On input $(\textit{passive corrupt}, i)$ on $F.\textit{infl}$:

- The functionality outputs $(\textit{state}, i, \sigma)$ on $F.\textit{leak}$.
- The value σ represents the internal state of party i .

The internal state includes:

- all previous inputs of the party,
- messages waiting in its input queue,
- previously generated outputs.

Thus the adversary learns everything the party knows but does not control its future behavior.

Active corruption gives the adversary full control over a party.

On input (*active corrupt*, i) on $F.infl$:

- The functionality records that party i is corrupted.
- The internal state of the party is leaked to the adversary.
- The functionality stops processing normal inputs and outputs for that party.

Instead, future actions involving that party are controlled by the adversary.

Behavior After Active Corruption

Once party i is actively corrupted, the adversary effectively controls its actions.

- Inputs intended for party i are handled through the influence port.
- Whenever the functionality would normally output a value y to party i ,
- it instead outputs $(output, i, y)$ on the leakage port.

This allows the adversary to observe and control all communication involving the corrupted party.

Interpretation of Corruption Models

Passive corruption corresponds to an adversary that only observes the party's internal data.

Active corruption corresponds to an adversary that completely controls the party.

After active corruption:

- the adversary chooses all future inputs for the corrupted party,
- the adversary learns all outputs generated for that party.

These corruption models allow the UC framework to capture realistic adversarial behavior.

Understanding the Ideal Functionality F_{ST}

The diagram for the secure transfer functionality F_{ST} represents how parties, the adversary, and the functionality interact.

$$\begin{aligned} \text{Parties} &\rightarrow ST.in_i \rightarrow F_{ST} \rightarrow ST.out_j \rightarrow \text{Parties} \\ \text{Adversary} &\leftarrow ST.leak \\ \text{Adversary} &\rightarrow ST.infl \end{aligned}$$

Protocol Ports

- $ST.in_i$: party i sends a message to the functionality.
- $ST.out_j$: the functionality delivers a message to party j .

Adversarial Ports

- $ST.leak$: information leaked to the adversary (e.g., sender, receiver, message length).
- $ST.infl$: commands from the adversary controlling message delivery.

Thus the functionality acts as a trusted message buffer while the adversary controls the scheduling of deliveries.

Protocols in the UC Model

A protocol in the UC framework consists of a set of parties

$$P_1, P_2, \dots, P_n$$

Each party is modeled as an interactive agent.

The protocol is written as

$$\pi = (P_1, P_2, \dots, P_n)$$

Each party communicates using named ports that determine how messages flow between the protocol, the network, and the adversary.

The goal of the protocol is to emulate the behavior of an ideal functionality F .

Ports of a Party

Each party in a protocol contains several ports.

Protocol Ports

- $P.in_i$: input from the ideal functionality
- $P.out_i$: output sent to the ideal functionality

These ports represent interaction with the functionality the protocol is trying to implement.

Resource Ports

- $R.in_i$
- $R.out_i$

These ports connect the party to the communication resource representing the network.

Special Ports

Each party also contains special ports used to model adversarial behavior.

- ① **Influence Port** $R.infl$; allows the adversary to influence communication
- ② **Leakage Port** $R.leak$; Outputs information that is visible to the adversary.

These ports are essential for modeling realistic network behavior and corruption.

Ideal Functionality for Authenticated Transfer

- 1 The ideal functionality F_{AT} models an authenticated communication channel between parties.
- 2 It ensures that messages are delivered without modification and that the sender identity cannot be forged.
- 3 However, the adversary may still control the order in which messages are delivered.

Behavior of F_{AT}

When a message is sent through the authenticated channel:

- On input (mid, i, j, m) on $AT.in_i$,
- The functionality outputs (mid, i, j, m) on $AT.leak$.
- The message is stored internally.

The adversary therefore learns the message contents but cannot modify them.

Message Delivery in F_{AT}

Delivery of messages is controlled through the influence port.

- On input $(deliver, mid, i, j)$ on $AT.infl$,
- If (mid, i, j, m) is stored, the functionality outputs (mid, i, m) on $AT.out_j$.

This models an authenticated but asynchronous channel where the adversary controls delivery timing.

Example Protocol for Secure Transfer

We now describe a protocol that implements secure message transfer using:

- an authenticated channel
- a public-key encryption scheme

Suppose party P_1 wants to send a message m to party P_2 .

Protocol Steps

The protocol proceeds as follows.

- 1 P_1 announces that it wants to send a message to P_2 .
- 2 P_2 generates a public-key encryption pair (ek, dk) .
- 3 P_2 sends the encryption key ek to P_1 .
- 4 P_1 encrypts the message

$$c = E_{ek}(m)$$

and sends c to P_2 .

- 5 P_2 decrypts

$$m = D_{dk}(c)$$

The behavior of the sender is modeled as follows.

- 1 On input $(mid, 2, m)$ on $ST.in_1$, output $(mid, 2, hello)$ on $AT.in_1$.
- 2 When receiving $(mid, 2, ek)$ on $AT.out_1$, sample encryption

$$c = E_{ek}(m)$$

and output $(mid, 2, c)$ on $AT.in_1$.

The receiver behaves as follows.

- 1 On input $(mid, 1, hello)$ on $AT.out_2$, generate key pair (ek, dk) and send $(mid, 1, ek)$.
- 2 On input $(mid, 1, c)$ on $AT.out_2$, compute

$$m = D_{dk}(c)$$

and output $(mid, 1, m)$ on $ST.out_2$.

Goal of the Protocol

- 1 The protocol aims to implement the ideal functionality for secure transfer F_{ST} .
- 2 The authenticated channel F_{AT} is used as a communication resource.
- 3 The protocol π_{ST} together with F_{AT} should behave like the ideal functionality F_{ST} .

Formally, we want to show

$$\pi_{ST} \circ F_{AT} \approx F_{ST}$$

Modeling Corruption

The UC model also specifies how corruption of parties is handled.

Two corruption types are considered.

- Passive corruption
- Active corruption

These models capture different levels of adversarial power over the parties.

Passive Corruption of a Party

If a party receives the symbol

(passive corrupt)

on its influence port, it returns its internal state to the adversary.

The internal state includes:

- randomness used so far
- all previous inputs and outputs
- messages stored in its queues

However, the party continues to execute the protocol honestly.

Active Corruption of a Party

If a party receives the symbol

(active corrupt)

then the adversary takes full control.

The party outputs its internal state and then follows only commands from the adversary.
From this point onward, the adversary controls all behavior of that party.

Protocol Execution Using a Resource

In the UC model, a protocol π does not communicate directly between parties. Instead it uses a communication resource R .

For example, if a party P_i wants to send a message m , it sends the command

$$(send, P, m)$$

on the port $R.infl_i$.

The resource R then delivers the message through the appropriate output port.

Similarly, a party may read messages using

$$(read, P)$$

which returns the next message stored for that party.

Thus communication between parties is mediated through the resource.

Corrupting a Party in a Protocol

Corruption of parties is also modeled through the resource.

Passive Corruption

- The adversary issues (*passive corrupt*, i).
- The party reveals its internal state.
- The party continues to follow the protocol honestly.

Active Corruption

- The adversary issues (*active corrupt*, i).
- The party reveals its internal state.
- The adversary takes full control of the party.

After active corruption, the adversary can send commands to the party using read and send operations.

The Simulator

When comparing a protocol π with an ideal functionality F , their port structures may differ. For example:

- The protocol π communicates using a resource R .
- The ideal functionality F has leakage and influence ports.

Because of these differences, the two systems cannot be directly compared. To solve this problem, the UC framework introduces a **simulator** S .

Role of the Simulator

- ① The simulator connects the adversary to the ideal functionality.
- ② Its purpose is to reproduce the behavior that the adversary would observe in the real protocol execution.

The simulator translates:

- adversarial commands
- leakage from the functionality
- influence on message delivery

into behavior consistent with the real protocol.

If the simulator succeeds, the environment cannot distinguish the real execution from the ideal execution.

Structure of the Ideal Execution

In the ideal world, the system consists of:

- the ideal functionality F
- the simulator S
- the environment Z

The simulator interacts with F through its leakage and influence ports.

From the perspective of the environment, the simulator plays the role of the adversary.

Thus the ideal system appears similar to the real protocol execution.

Simulator Properties

The simulator must satisfy several requirements.

Polynomial Time

The simulator must run in expected polynomial time.

Corruption Preserving

If a party is corrupted in the real protocol, the simulator must corrupt the corresponding party in the ideal functionality.

Clock Preservation

The simulator must preserve the activation structure of the system.

These requirements ensure that the simulator faithfully represents adversarial behavior.

Example Composition with Simulator

In the real execution we have

$$\pi_{ST} \circ F_{AT}$$

which represents the protocol using an authenticated transfer functionality.

In the ideal execution we have

$$F_{ST} \circ S$$

where S is the simulator.

The security goal is that these two systems behave indistinguishably.

The Environment

To formally compare the real and ideal systems, the UC framework introduces an **environment Z** .

The environment interacts with the system through open ports.

It can:

- provide inputs to parties
- receive outputs from parties
- interact with the adversary

The goal of the environment is to distinguish between the real and ideal executions.

Two systems are considered indistinguishable if no environment can tell them apart.
Formally,

$$\pi \circ R \approx F \circ S$$

This means that for every environment Z , the outputs of the two systems are computationally indistinguishable.

If this holds, we say that the protocol **UC-realizes** the ideal functionality.

Open and Closed Systems

An interactive system may have open ports.

Open ports allow communication with the environment.

When the environment is connected to all open ports, the system becomes a **closed system**.

A closed system has no remaining open ports.

Security comparisons are made on closed systems, since the environment is fully connected.

Intuition Behind the UC Security Definition

The UC security definition follows this idea:

- 1 Consider the real system

$$\pi \circ R$$

- 2 Consider the ideal system

$$F \circ S$$

- 3 Connect both systems to the same environment Z .
- 4 If Z cannot distinguish them, the protocol securely implements F .

Thus the protocol behaves exactly like the ideal functionality from the viewpoint of any external observer.



